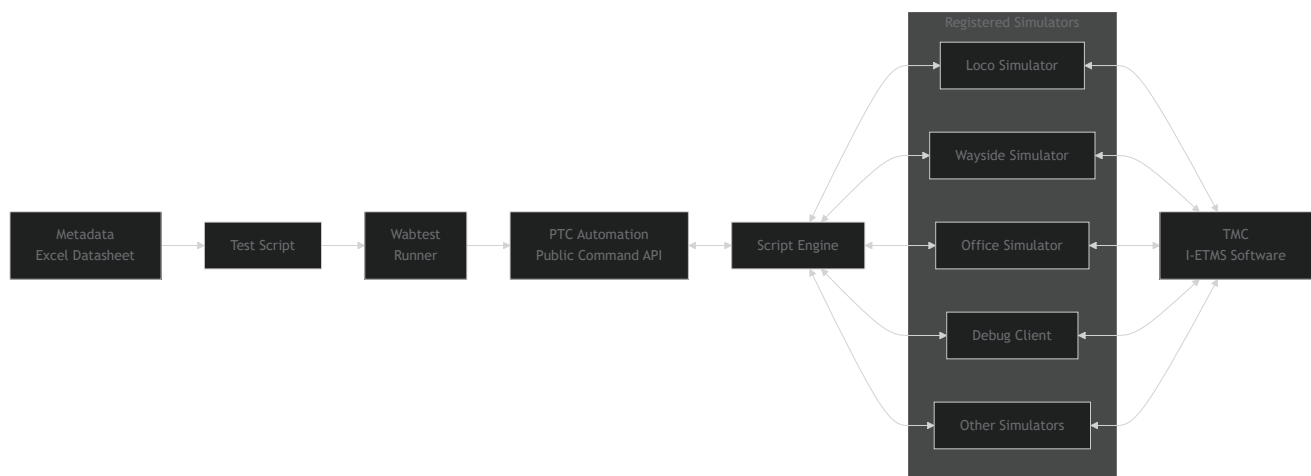


Writing WCR test scripts

Test scripts are written in Python and use the public WCR API to interact with the simulators and TMC. The scripts can be automated or manual, depending on the execution type set at the beginning of the script. Datasheets are used to provide test metadata which defines the test cases. The Wabtest runner executes the test python test scripts, manages reporting, and invokes the appropriate test-case handlers based on the execution type.

WCR Test Execution Architecture

The following diagram shows how test metadata, Wabtest, the PTC Automation framework, simulators, and the TMC interact during test execution.



Component Roles

- **Metadata (Excel Datasheet):** Contains the test-case metadata and input data used by Wabtest during execution. Typical information includes test case identifiers, scenario definitions, execution types, configuration values, expected results, requirements references, and other parameters accessed through the `data_sheet` object passed to test-case handlers.
- **Test Script:** User-authored WCR automation or manual test script that imports and calls the public API (`wcr_*` objects). The script uses information from the metadata datasheet to determine which scenarios to execute and what data to apply.
- **Wabtest Runner:** Loads the test metadata, executes test cases, manages reporting, invokes test-case handlers, and passes the current datasheet row (`data_sheet`) to the script.
- **PTC Automation:** Provides the public automation commands exposed through `Common.WCR_public` . This layer abstracts the underlying simulator and system interactions behind a stable public API.
- **Script Engine:** Registers and manages all available simulators and routes automation requests from PTC Automation to the appropriate simulator or subsystem.
- **Simulators:** Components such as Loco, Wayside, Office, Debug Client, and other environment simulators used to emulate external systems and generate messages, state changes, and test stimuli.

- **TMC (I-ETMS Software):** The Train Management Computer running the I-ETMS software under test. Simulators communicate with the TMC to exchange messages and verify system behavior.

Script structure

WCR test scripts import the public test API with:

```
from Common.WCR_public import *
```

The public objects use the `wcr_` prefix and other naming conventions. Their methods accept the same arguments as the simulator library and are documented in `Common/WCR_public.pyi`. The stub is generated from the private implementation docstrings; regenerate it after API or documentation changes:

```
python tools\generate_wcr_stubs.py
```

An automated script normally sets the execution type, registers a test-case handler, and starts the runner. There are more options available options described in the `run_test` module documentation.

```
from Common.WCR_public import *

def main():
    run_test.set_execution_type("Automation")
    run_test.set_test_case_function_table({
        "Positive": "handle_test_case",
        "Negative": "handle_test_case",
    })
    run_test.run()

def handle_test_case(data_sheet=None):
    case = data_sheet["S_no"]
    if case == 1:
        wcr_loco_sim.set_position({
            "sub_id": 8880,
            "track_group": "Main",
            "point": 6,
            "direction": "Increasing",
            "stealth": True,
        })
        wcr_state.set_onboard_state({"Next_State": "ACTIVE"})
        wcr_ptc.verify_state({
            "OB_State": "ACTIVE",
            "expected_result": True,
            "req_verified": "On-board is active",
        })

if __name__ == "__main__":
    main()
```

`data_sheet` is the current row from the test data source. Use its `s_no` value to select a scenario. Public methods are called on the corresponding `wcr_` object, for example:

```
wcr_debug_client.set_tbc({
    "scac": "COMMON",
    "parameter": "TBC177",
    "value": 2,
    "expected_result": True,
})
wcr_office_base.reset_msg(2070)
time_text = wcr_office_base.get_value({
    "aMsgID": 2070,
    "aField": "Time",
    "timeout_sec": 120,
})
wcr_utility.report_to_qa({
    "data1": 1,
    "data2": 2,
    "operation": "<=",
    "expected_result": True,
    "req_verified": "The requirement is met",
})
```

Manual scripts use the same structure but set `"Manual"` and use the user action commands to record or observe operator behavior:

```
def main():
    run_test.set_execution_type("Manual")
    run_test.set_test_case_function_table({
        "Positive": "handle_test_case",
        "Negative": "handle_test_case",
    })
    run_test.run()

def handle_test_case(data_sheet=None):
    wcr_start_up.automation_setup()
    wcr_state.set_onboard_state({"Next_State": "ACTIVE"})
    wcr_user_act.input_verification("The active state is displayed")
    wcr_user_act.observe("Observe the expected track display")
    wcr_state.set_onboard_to_cutout_via_1010_msg()
```

Keep simulator setup and preconditions before the scenario, reset messages before reading them, and clean up changed configuration at the end of a test case. Use the generated stub or your editor's signature help to see each command's parameters, return value, and implementation documentation.

Metadata Datasheet

The datasheet is structured with columns representing different parameters and rows representing individual test cases. Each row corresponds to a specific test case, and the columns provide the necessary information for executing that test case. The datasheet is used to drive the execution of test scripts, allowing for easy management and organization of

test cases. It enables testers to define various scenarios, input data, and expected outcomes, facilitating comprehensive testing of the system under test.

Parameters are accessed through the `data_sheet` object passed to test-case handlers.

Typical information includes (shown in the image below):

- [S_no]: Test case identifier
- [Requirement]: Requirement reference numbers
- [Description]: Scenario definitions
- [Folder]: Name of the folder the test script is located in, defined in RQM as the Category
- [Test_Type]: The test type (Positive or Negative) defined in RQM as the Test Type
- [Test_Technique]: The test technique describes the approach used to design the test case. The only valid values are:
 - [Boundary Value Analysis]: A test technique that focuses on testing the boundaries or limits of an object to identify potential issues or defects.
 - [Equivalence Partitioning]: A test technique that divides the input data into equivalent classes
 - [Decision Table Testing]: A test technique that uses a decision table to represent different combinations of inputs and their corresponding expected outputs.
 - [State Transition Testing]: A test technique that focuses on testing the behavior of a system based on its different states and transitions between those states.
 - [Error Guessing]: A test technique that relies on the tester's experience and intuition to identify potential error-prone areas in the system.
 - [All Pairs Testing]: A test technique that tests all possible combinations of input values to ensure comprehensive coverage.
 - [Cause Effect Testing]: A test technique that identifies the causes of potential problems and their effects on the system to design test cases.
- [Retired] Indicates whether the test case is retired and should no longer be executed.
- [Scorable] Indicates whether the test case is scorable. Scorable test cases are those that can be validated and reported to RQM as pass or fail.

Example Datasheet Structure

	A	B	C	D	E	F	G	H	I
1	S_no	Requirement	Description	Folder	Test_Type	Test_Technique	Retired?	Scorable	Comments
2	1	L2R454612	initialization -TBC137 for UP is set to 19 MPH - Onboard does not have any known locomotive track location - Onboard enforces for restricted speed then Verify onboard uses the value of TBC137(19) defined in the selected operating railroad(UP) configuration fileset	Restricted State	Positive	Equivalence Partitioning	False	Yes	Out of range values test for TBC137 have been covered in L2R6808
3	2	L2R454612	While onboard state is Restricted, UP, BNSF and NS were selected as operating railroad during last initialization - TBC137 is set to 20, 5 and 15 MPH for UP, BNSF and NS - Onboard does not have any known locomotive track location - Onboard enforces for restricted speed then Verify onboard uses the lowest value of TBC137(5) defined in the railroad specific configuration filesets of the operating railroads selected for initialization(UP, BNSF and NS)	Restricted State	Positive	Equivalence Partitioning	False	Yes	
4	3	L2R454612	While onboard state is Restricted, UP, BNSF and NS were selected as operating railroad during last initialization - TBC137 is set to 20, 0 and 10 MPH for UP, BNSF and NS - Onboard has known locomotive track location(Leading Edge Located in NS subdivision) - Onboard enforces for restricted speed then Verify onboard uses the value of TBC137(10) defined in the railroad specific configuration fileset of the operating railroad(NS) for the subdivision/district(1033) in which the leading edge of the train was located	Restricted State	Negative	Boundary Value Analysis	False	No	Covered in L2R11046